

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ  
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №4  
по курсу «Алгоритмы и структуры данных»

Тема: Стек, очередь, связанный список.

Вариант 7 (Продвинутый уровень)

Выполнила:

Кулакова Елизавета Алексеевна

К3260

Проверил:

Харлуниин А. А.

Санкт-Петербург

2025 г.

## Содержание отчета

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Содержание отчета.....</b>                                  | <b>2</b>  |
| <b>Задачи по варианту (базовый уровень).....</b>               | <b>3</b>  |
| Задача №1. Стек.....                                           | 3         |
| Задача №4. Скобочная последовательность. Версия 2.....         | 10        |
| Задача №5. Стек с максимумом.....                              | 17        |
| Задача №7. Максимум в движущейся последовательности.....       | 26        |
| <b>Задачи на продвинутый уровень.....</b>                      | <b>32</b> |
| Задача №12. Строй новобранцев.....                             | 32        |
| Задача №13. Реализация стека, очереди и связанных списков..... | 41        |
| Пункт 3. Односвязный список.....                               | 41        |
| Пункт 4. Двусвязный список.....                                | 44        |
| <b>Вывод.....</b>                                              | <b>49</b> |

## Задачи по варианту (базовый уровень)

### Задача №1. Стек

Реализуйте работу стека. Для каждой операции изъятия элемента выведите ее результат.

На вход программе подаются строки, содержащие команды. Каждая строка содержит одну команду. Команда — это либо “+  $N$ ”, либо “-”. Команда “+  $N$ ” означает добавление в стек числа  $N$ , по модулю не превышающего  $10^9$ . Команда “-” означает изъятие элемента из стека. Гарантируется, что не происходит извлечения из пустого стека. Гарантируется, что размер стека в процессе выполнения команд не превысит  $10^6$  элементов.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится  $M$  ( $1 \leq M \leq 10^6$ ) – число команд. Каждая последующая строка исходного файла содержит ровно одну команду.
- **Формат выходного файла (output.txt).** Выведите числа, которые удаляются из стека с помощью команды “-”, по одному в каждой строке. Числа нужно выводить в том порядке, в котором они были извлечены из стека. Гарантируется, что изъятий из пустого стека не производится.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

| input.txt | output.txt |
|-----------|------------|
| 6         | 10         |
| + 1       | 1234       |
| + 10      |            |
| -         |            |
| + 2       |            |
| + 1234    |            |
| -         |            |

### Листинг кода

```
from stack_class import *
from generate_commands import *
from measurments import *

@measure_performance
def stack_flow(n, cmnds):
    my_stack = Stack(10**6)
    popped = []
    for cmd in cmnds:
        if cmd == '-':
            popped.append(my_stack.pop())
        else:
            val = int(cmd.strip('+ '))
            my_stack.push(val)
    return popped

if __name__ == "__main__":
    generate_cmnds(10**6)
```

```

with open('input.txt') as f:
    n = int(f.readline())
    commands = []
    for i in range(n):
        commands.append(f.readline().strip())

res = stack_flow(n, commands)
with open('output.txt', 'w') as f:
    for r in res:
        f.write(f'{str(r)}\n')

```

## Листинг кода функции для формирования массива и класса стека

```

class Stack:
    def __init__(self, size):
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        self.top += 1
        self.stack[self.top] = item

    def pop(self):
        if self.is_empty():
            return None
        else:
            popped = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            return popped

    def is_empty(self) -> bool:
        if self.top == -1:
            return True
        return False

    def __getitem__(self, index):
        return self.stack[index]

```

```

import random
import numpy as np

def generate_cmds(size=None):
    if size is None:
        size = random.randint(1, 10**6)

    cmds = []
    stack_size = 0

    for _ in range(size):
        # если стек пуст, можем только пушнуть
        if stack_size == 0:
            x = random.randint(-10 ** 9, 10 ** 9)
            cmds.append(f"+ {x}")

```

```

        stack_size += 1
    else:
        # иначе случайно выбираем, пуш или поп
        if random.random() < 0.75: # вероятность пуша
            x = random.randint(-10 ** 9, 10 ** 9)
            cmds.append(f"+ {x}")
            stack_size += 1
        else:
            cmds.append("-")
            stack_size -= 1

    with open('input.txt', 'w') as f:
        f.write(str(size) + "\n")
        f.write("\n".join(cmds))

    return None

```

### Текстовое объяснение решения.

В этой задаче нужно реализовать самый базовый стек и «прогнать» по нему последовательность команд. Вход задаётся так: сначала число  $M$  - количество команд, затем  $M$  строк вида "+ N" (положить число  $N$  на стек) или "-" (снять верхний элемент). На каждую команду извлечения мы обязаны вывести значение, которое снимается со стека, в том порядке, в котором оно извлекалось. По условию гарантируется, что извлечения из пустого стека не будет, а размер стека не превысит  $10^6$ .

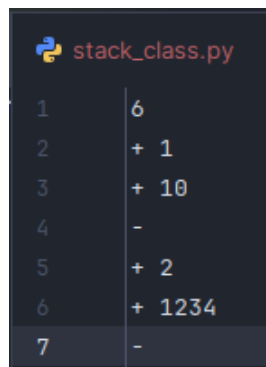
Для этой задачи был реализован отдельный класс `Stack`, максимально близкий к «ручной» реализации стека. Внутри он хранит заранее выделенный массив фиксированного размера и индекс вершины стека. В конструкторе создаётся список заданной длины, заполненный `None`, а переменная `top` инициализируется значением `-1`, что соответствует пустому стеку. Метод `push` при добавлении элемента сначала сдвигает вершину на одну позицию вверх, а затем записывает значение по этому индексу; метод `pop` возвращает элемент с вершины: берёт значение по индексу `top`, обнуляет эту ячейку (ставит `None`) и сдвигает вершину вниз. Метод `is_empty` сравнивает `top` с `-1`, чтобы быстро проверить, пуст ли стек, а `__getitem__` позволяет при необходимости обращаться к внутреннему массиву по индексу. Такая реализация явно моделирует структуру данных стек: элементы размещаются в заранее выделенной области памяти, а все операции `push/pop` работают за  $O(1)$  и строго следуют принципу LIFO.

Основная функция `stack_flow` принимает количество команд и список строк-команд. В начале я создаю стек и пустой список `porped` для накопления ответов. Затем просто прохожу по всем командам по очереди: если строка равна "-", вызываю `my_stack.pop()` и добавляю результат в `porped`; если команда начинается с "+", извлекаю из строки число и кладу

его в стек через push. По условию вход корректен, поэтому дополнительных проверок на пустой стек не требуется. После выполнения всех команд список извлечённых значений записывается в output.txt, по одному числу в строке - это именно тот формат, который требуется задачей.

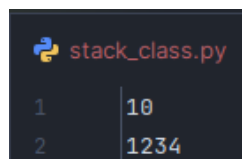
Для генерации тестовых наборов команд я использовала функцию generate\_cmds: она случайным образом чередует операции + N и -, при этом внутри ведёт счётчик текущего размера стека и просто не генерирует -, когда стек пуст. Это гарантирует, что вход всегда корректен и не содержит изъятий из пустого стека, как и обещано в условии.

### Результат работы кода:



```
stack_class.py
1      6
2      + 1
3      + 10
4      -
5      + 2
6      + 1234
7      -
```

Рисунок 1 – Файл input.txt с входными данными из примера



```
stack_class.py
1      10
2      1234
```

Рисунок 2 – Результат работы функции, пример из задания

```

stack_class.py 1-st
1      68652
2      + 906424886
3      + -165620983
4      + 383462200
5      + 732268251
6      + 372451832
7      + -579563547
8      -
9      + 42949942
10     + -626360689
11     + 384038237
12     + -979945627
13     + 409029740
14     + -605689608
15     + -317575460
16     + -54664393
17     + -152466198
18     + -380238461
19     + -801446375
20     + -940578226
21     + -772057199
22     -
23     + -909907383
24     -
25     + -590517330

```

Рисунок 3 – Файл input.txt со сгенерированными входными данными (68652 команды)

```

stack_class.py 1-
1      -579563547
2      -772057199
3      -909907383
4      274427530
5      -278054140
6      -901622278
7      -481039518
8      527842144
9      -192646802
10     191433968
11     671589092
12     329112403
13     791636694
14     -293761460
15     -196300052
16     823232993
17     -702201643
18     399436974
19     540004694
20     788624952
21     -15445123
22     227412336
23     -330364940
24     -216603009
25     798886533

```

Рисунок 4 – Результат работы функции, сгенерированные входные данные

**Результат работы кода на максимальных и минимальных значениях:**

```

stack_class.py
1 | 1
2 | + -183564388

```

Рисунок 5 – Файл input.txt с вводными данными – минимальными значениями

```

stack_class.py  1-stack_flow.py  output.txt x
1

```

Рисунок 6 – Файл output.txt после выполнения функции с минимальными значениями

```

stack_class.py  1-stack
⚠ The file size (9.79 MB) exceeds
1 | 1000000
2 | + -626855839
3 | -
4 | + -487974729
5 | + 846152368
6 | + 7653945
7 | + 756097023
8 | + 878365745
9 | + 618946472
10 | + 271563650
11 | + 30216082
12 | + -798331212
13 | + 871590052
14 | + 264018634
15 | + -779758111
16 | + -457653724
17 | + -724754473
18 | + 959547544
19 | + -325073636
20 | + -303740980
21 | + 726150571
22 | -
23 | + 461308790

```

Рисунок 7 – Файл input.txt с вводными данными – максимальными значениями

```

stack_class.py 1-stack
⚠ The file size (2.6 MB) exceeds

1 -626855839
2 726150571
3 -791819843
4 -46561349
5 19736570
6 -447323939
7 -529619208
8 65105048
9 752338314
10 224635220
11 -568804513
12 331522403
13 -878298679
14 -284209477
15 -173374290
16 233456727
17 -106463803
18 -669054307
19 -790457505
20 -975769884
21 -11763835
22 -780503632
23 912628011
24 021502011

```

Рисунок 8 – Файл output.txt после выполнения функции с максимальными значениями

**Таблица затрат памяти и времени по заданию 1:**

|                                                                    | Время выполнения | Затраты памяти |
|--------------------------------------------------------------------|------------------|----------------|
| Нижняя граница диапазона значений входных данных из текста задачи  | 0.0028 сек       | 8.58 МВ        |
| Сгенерированный пример                                             | 0.0353 сек       | 5.49 КВ        |
| Пример из задания                                                  | 0.0029 сек       | 10.09 МВ       |
| Верхняя граница диапазона значений входных данных из текста задачи | 0.5125 сек       | 34.21 МВ       |

### Вывод по задаче:

По сложности решение линейное: мы один раз читаем  $M$  строк, для каждой делаем  $O(1)$  операцию со стеком и, при необходимости, пишем результат. Время работы  $O(M)$ , память —  $O(M)$  в худшем случае, когда стек почти не опустошается. При ограничении  $M \leq 10^6$  и разумной реализации стека на списке Python это укладывается в заданные 2 секунды и 256 МБ.

## Задача №4. Скобочная последовательность. Версия 2

Определение правильной скобочной последовательности такое же, как и в задаче 3, но теперь у нас больше набор скобок: `[]{}()`.

Нужно написать функцию для проверки наличия ошибок при использовании разных типов скобок в текстовом редакторе типа LaTeX.

Для удобства, текстовый редактор должен не только информировать о наличии ошибки в использовании скобок, но также указать точное место в коде (тексте) с ошибочной скобочкой.

В первую очередь объявляется ошибка при наличии первой несовпадающей закрывающей скобки, перед которой отсутствует открывающая скобка, или которая не соответствует открывающей, например, `()[]` - здесь ошибка укажет на `]`.

Во вторую очередь, если описанной выше ошибки не было найдено, нужно указать на первую несовпадающую открывающую скобку, у которой отсутствует закрывающая, например, `( [ ]`.

Если не найдено ни одной из указанных выше ошибок, нужно сообщить, что использование скобок корректно.

Помимо скобок, код может содержать большие и маленькие латинские буквы, цифры и знаки препинания.

Формально, все скобки в коде (тексте) должны быть разделены на пары совпадающих скобок, так что в каждой паре открывающая скобка идет перед закрывающей скобкой, а для любых двух пар скобок одна из них вложена внутри другой, как в `foo[bar]` или они разделены, как в `f(a, b) - g[c]`. Скобка `[` соответствует скобке `]`, соответствует `(` скобке `)`.

- **Формат входного файла (input.txt).** Входные данные содержат одну строку  $S$ , состоящую из больших и маленьких латинских букв, цифр, знаков препинания и скобок из набора `[]{}()`. Длина строки  $S - 1 \leq S \leq 10^5$ .
- **Формат выходного файла (output.txt).** Если в строке  $S$  скобки используются правильно, выведите «Success» (без кавычек). В противном случае выведите отсчитываемый от 1 индекс первой несовпадающей закрывающей скобки, а если нет несовпадающих закрывающих скобок, выведите отсчитываемый от 1 индекс первой открывающей скобки, не имеющей закрывающей.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

| input.txt                | output.txt |
|--------------------------|------------|
| <code>[]</code>          | Success    |
| <code>[][]</code>        | Success    |
| <code>[()]</code>        | Success    |
| <code>(())</code>        | Success    |
| <code>{</code>           | 1          |
| <code>{[]</code>         | 3          |
| <code>foo(bar);</code>   | Success    |
| <code>foo(bar[i];</code> | 10         |

### Листинг кода.

```
from stack_class import *
from measurments import *
from random_string import *

@measure_performance
def check_brackets(text):
```

```

open_brackets = ['[', '{', '(']
close_brackets = [']', '}', ')']
brackets = open_brackets + close_brackets
size = len(text)
stack = Stack(size) # макс размер ставим, так как все скобки
открывающие могут быть
for i in range(size):
    el = text[i]
    if el in brackets:
        if el in close_brackets:
            if stack.is_empty():
                return i+1
            else:
                if open_brackets.index(stack[stack.top][0]) !=
close_brackets.index(el):
                    return i+1
                else:
                    stack.pop() # удаляем, так как нашлась пара
                    continue
            stack.push((el, i+1))
        continue
    if stack.is_empty():
        return 'Success'
    else:
        return stack[stack.top][1]

if __name__ == "__main__":
    generate_string(10**5)
    with open('input.txt') as f:
        line = f.readline()

    result = check_brackets(line)
    with open('output.txt', 'w') as f:
        f.write(str(result))
    print(result)

```

### Листинг кода дополнительных функций.

```

import random
import string

def generate_string(length=None):
    chars = string.ascii_letters + string.digits + string.punctuation
    brackets = '()[{}]'

    if length is None:
        length = random.randint(1, 10 ** 5)

    str_list = [random.choice(chars) for _ in range(length)]

    if length > 1:
        num_brackets = random.randint(1, length // 2)
    else:
        num_brackets = 1

    positions = random.sample(range(length), num_brackets)

```

```

for pos in positions:
    str_list[pos] = random.choice(brackets)

result = ''.join(str_list)

with open('input.txt', 'w') as f:
    f.write(result)

return None

```

### Текстовое объяснение решения.

Идея решения опирается на стек. Я завожу два списка: `open_brackets = ['[', '{', '(']` и `close_brackets = [']', '}', ')']`, а также стек фиксированного размера, равного длине строки (это максимум возможного числа открывающих скобок). Строка просматривается слева направо, и каждый раз, когда встречается скобка, принимается локальное решение: если это открывающая скобка, она запоминается в стеке вместе с позицией в строке; если закрывающая - проверяется, есть ли ей подходящая пара на вершине стека. Благодаря такому пошаговому разбору мы в любой момент знаем текущее состояние «незакрытых» скобок и можем сразу поймать первую ошибку при закрытии - либо стек пуст (нет пары), либо типы скобок не совпадают. Все остальные символы (буквы, цифры, знаки препинания) пропускаются, они на логику скобочной последовательности не влияют.

Когда строка заканчивается, у нас два варианта. Если стек пуст, значит, каждая открывающая нашла свою закрывающую - скобки корректны, и функция возвращает "Success". Если же в стеке что-то осталось, то по условию нужно указать позицию первой открывающей скобки без пары. Благодаря тому, что в стек складывались пары (скобка, позиция) и мы никогда не трогаем более ранние элементы, в вершине сейчас лежит последняя открывающая, первой «незакрытой» будет как раз самая верхняя из оставшихся. В реализации берётся верхняя (`stack[stack.top][1]`), это и будет первая открывающая, у которой не нашлось пары.

Для генерации тестовых строк я использовала отдельную функцию `generate_string`, которая создаёт случайную строку из латинских букв, цифр и знаков препинания, а затем обязательно подмешивает в неё несколько случайных скобок из набора `()[]{}.`, чтобы было что проверять. Это помогает обкатать алгоритм на разных длинах и конфигурациях строк, в том числе на максимальной по условию длине  $10^5$ .

## Результат работы кода на примерах из задания:

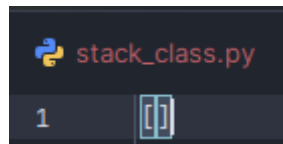


Рисунок 9 – Файл input.txt с данными примера 1

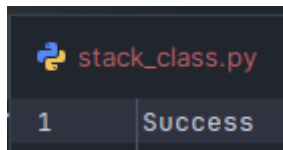


Рисунок 10 – Файл output.txt с результатом работы на данных примера 1

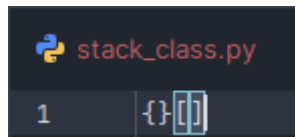


Рисунок 11 – Файл input.txt с данными примера 2

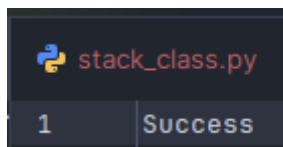


Рисунок 12 – Файл output.txt с результатом работы на данных примера 2

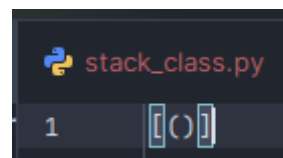


Рисунок 13 – Файл input.txt с данными примера 3

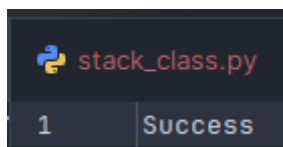


Рисунок 14 – Файл output.txt с результатом работы на данных примера 3

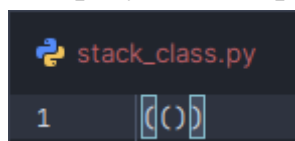


Рисунок 15 – Файл input.txt с данными примера 4

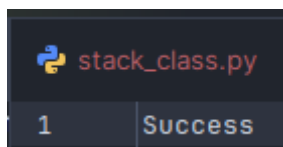


Рисунок 16 – Файл output.txt с результатом работы на данных примера 4

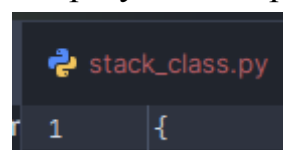


Рисунок 17 – Файл input.txt с данными примера 5

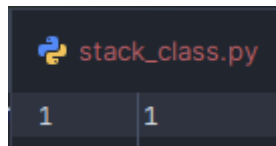


Рисунок 18 – Файл output.txt с результатом работы на данных примера 5

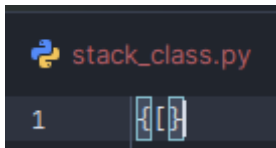


Рисунок 19 – Файл input.txt с данными примера 6

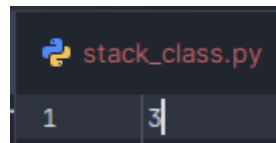


Рисунок 20 – Файл output.txt с результатом работы на данных примера 6

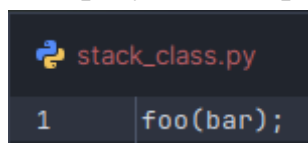


Рисунок 21 – Файл input.txt с данными примера 7

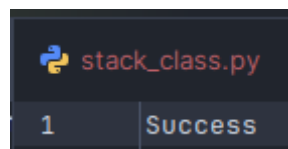


Рисунок 22 – Файл output.txt с результатом работы на данных примера 7

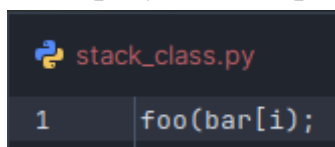


Рисунок 23 – Файл input.txt с данными примера 8

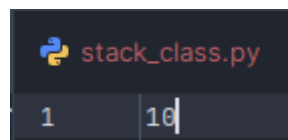


Рисунок 24 – Файл output.txt с результатом работы на данных примера 8

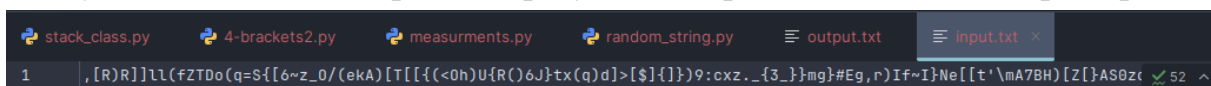


Рисунок 25 – Файл input.txt с данными сгенерированного примера (39594 элементов)

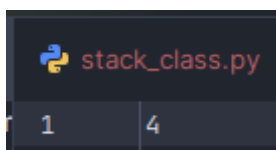


Рисунок 26 – Файл output.txt с результатом работы на данных сгенерированного примера

**Результат работы кода на максимальных и минимальных значениях:**

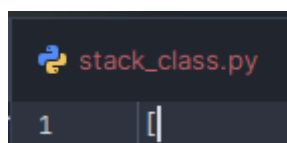


Рисунок 27 – Файл input.txt с вводными данными – минимальными значениями

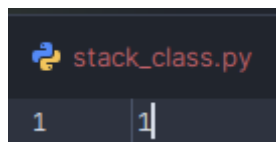


Рисунок 28 – Файл output.txt после выполнения функции с минимальными значениями



Рисунок 29 – Файл input.txt с вводными данными – максимальными значениями

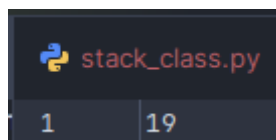


Рисунок 30 – Файл output.txt после выполнения функции с максимальными значениями

#### **Таблица затрат памяти и времени по заданию 4:**

|                                                                   | Время выполнения | Затраты памяти |
|-------------------------------------------------------------------|------------------|----------------|
| Нижняя граница диапазона значений входных данных из текста задачи | 0.0000 сек       | 0.51 KB        |
| Пример 1                                                          | 0.0000 сек       | 0.58 KB        |
| Пример 2                                                          | 0.0000 сек       | 0.59 KB        |
| Пример 3                                                          | 0.0000 сек       | 0.59 KB        |
| Пример 4                                                          | 0.0000 сек       | 0.59 KB        |
| Пример 5                                                          | 0.0000 сек       | 0.57 KB        |
| Пример 6                                                          | 0.0000 сек       | 0.59 KB        |

|                                                                             |            |           |
|-----------------------------------------------------------------------------|------------|-----------|
| Пример 7                                                                    | 0.0000 сек | 0.63 KB   |
| Пример 8                                                                    | 0.0000 сек | 0.65 KB   |
| Собственный пример                                                          | 0.0000 сек | 309.89 KB |
| Верхняя граница<br>диапазона значений<br>входных данных из<br>текста задачи | 0.0001 сек | 781.87 KB |

**Вывод по задаче:** стек здесь идеально подходит, потому что сам по себе реализует нужную вложенность скобок. Мы проходим по строке один раз, операции push и pop выполняются за  $O(1)$ , поэтому общая сложность алгоритма -  $O(n)$  по времени и  $O(n)$  по памяти, что легко укладывается в заданные ограничения. При этом мы не только проверяем правильность скобочной последовательности, но и можем точно указать первую проблемную позицию — либо закрывающую без пары/не того типа, либо открывающую, у которой не нашлось пары до конца строки.

## Задача №5. Стек с максимумом

Стек - это абстрактный тип данных, поддерживающий операции `Push()` и `Pop()`. Нетрудно реализовать его таким образом, чтобы обе эти операции работали за константное время. В этой задаче ваша цель - реализовать стек, который также поддерживает поиск максимального значения и гарантирует, что все операции по-прежнему работают за константное время.

Реализуйте стек, поддерживающий операции `Push()`, `Pop()` и `Max()`.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится  $n$  ( $1 \leq n \leq 400000$ ) – число команд. Последующие  $n$  строк исходного файла содержит ровно одну команду: `push V`, `pop` или `max`.  $0 \leq V \leq 10^5$ .
- **Формат выходного файла (output.txt).** Для каждого запроса `max` выведите (в отдельной строке) максимальное значение стека.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.
- Пример:

| input.txt | output.txt | input.txt | output.txt | input.txt | output.txt |
|-----------|------------|-----------|------------|-----------|------------|
| 5         | 2          | 5         | 2          | 3         |            |
| push 2    | 2          | push 1    | 1          | push 1    |            |
| push 1    |            | push 2    |            | push 7    |            |
| max       |            | max       |            | pop       |            |
| pop       |            | pop       |            |           |            |
| max       |            | max       |            |           |            |

| input.txt | output.txt | input.txt | output.txt |
|-----------|------------|-----------|------------|
| 10        | 9          | 6         | 7          |
| push 2    | 9          | push 7    | 7          |
| push 3    | 9          | push 1    |            |
| push 9    | 9          | push 7    |            |
| push 7    |            | max       |            |
| push 2    |            | pop       |            |
| max       |            | max       |            |
| max       |            |           |            |
| max       |            |           |            |
| pop       |            |           |            |
| max       |            |           |            |

### Листинг кода.

```
from stack_class import *
from random_commands import *
from measurments import *

@measure_performance
def stack_mix(n, cmnds):
    my_stack = Stack(n)
    maxs = []
    for cmdnd in cmnds:
        if cmdnd == 'max':
            maxs.append(my_stack.max)
        else:
            if cmdnd == 'pop':
                my_stack.pop()
            else:
```

```

        val = int(cmnd.strip('push '))
        my_stack.push(val)

    return maxs

if __name__ == "__main__":
    generate_cmnds(4 * 10**5)
    with open('input.txt') as f:
        n = int(f.readline())
        commands = []
        for i in range(n):
            commands.append(f.readline().strip())

    res = stack_mix(n, commands)
    with open('output.txt', 'w') as f:
        for r in res:
            f.write(f'{str(r)}\n')
    print(res)

```

## Листинг кода дополнительных функций.

```

class Stack:
    def __init__(self, size):
        self.stack = [None] * size
        self.top = -1
        self.maxs = []
        self.max = float('-inf')

    def push(self, item):
        if item >= self.max:
            self.max = item
            self.maxs.append(item)
        self.top += 1
        self.stack[self.top] = item

    def pop(self):
        if self.is_empty():
            return None
        else:
            popped = self.stack[self.top]
            if popped == self.max:
                self.maxs.pop()
                if not self.maxs:
                    self.max = float('-inf')
            else:
                self.max = self.maxs[-1]
            self.stack[self.top] = None
            self.top -= 1
            return popped

    def is_empty(self) -> bool:
        if self.top == -1:
            return True
        return False

    def __getitem__(self, index):
        return self.stack[index]

```

```

import random

def generate_cmds(size=None):
    if size is None:
        size = random.randint(1, 4 * 10**5)

    cmds = []
    stack_size = 0

    for _ in range(size):
        if stack_size == 0:
            x = random.randint(0, 10 ** 5)
            cmds.append(f"push {x}")
            stack_size += 1
        else:
            if random.random() < 0.75:
                x = random.randint(0, 10 ** 5)
                cmds.append(f"push {x}")
                stack_size += 1
            else:
                if random.random() < 0.5:
                    cmds.append("pop")
                    stack_size -= 1
                else:
                    cmds.append("max")

    with open('input.txt', 'w') as f:
        f.write(str(size) + "\n")
        f.write("\n".join(cmds))

    return None

```

### Текстовое объяснение решения.

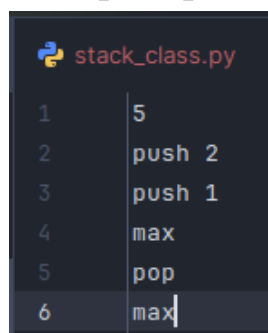
В этой задаче нужно реализовать стек, который помимо обычных операций push и pop умеет за  $O(1)$  отвечать на запрос max - то есть в любой момент времени возвращать текущий максимальный элемент в стеке. Перебирать элементы при каждом запросе нельзя: при  $n$  до 400 000 это сразу превратилось бы в  $O(n^2)$ .

Я решила это с помощью «стека максимумов». В классе Stack помимо основного массива для элементов и индекса вершины top я храню ещё список maxs и переменную max, в которой лежит текущий максимум. При push нового элемента сначала проверяется: если этот элемент не меньше текущего максимума, я обновляю self.max и добавляю его в maxs. Затем кладу элемент в основной стек. При pop снимаю верхний элемент; если он совпадает с текущим максимумом, значит, мы удаляем как раз «верхний» максимум - тогда я удаляю последний элемент из maxs и обновляю self.max: либо на предыдущий максимум из maxs[-1], либо в -inf,

если стек стал пуст. Таким образом, `maxs` хранит историю максимумов по мере роста стека, а при откате по `pop` максимум тоже корректно откатывается назад.

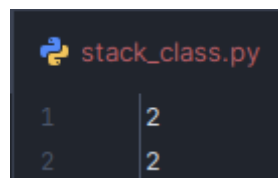
Функция `stack_mix` просто читает команды, применяет их к стеку и для каждой команды `max` записывает текущее значение `my_stack.max` в список результатов. Генератор команд `generate_cmnds` следит за тем, чтобы не вызывать `pop` на пустом стеке, и добавляет случайные `push/pop/max` в пределах ограничений. В итоге все три операции - `push`, `pop` и запрос максимума - работают за константное время, а общая сложность обработки  $n$  команд -  $O(n)$  по времени и  $O(n)$  по памяти, что уверенно укладывается в заданные лимиты даже на максимальных тестах.

### Результат работы кода на примерах из задания:



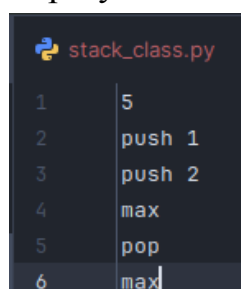
```
stack_class.py
1      5
2      push 2
3      push 1
4      max
5      pop
6      max
```

Рисунок 31 – Файл `input.txt` с данными примера 1



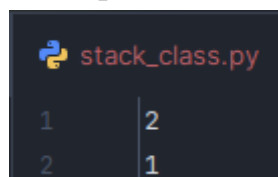
```
stack_class.py
1      2
2      2
```

Рисунок 32 – Файл `output.txt` с результатом работы на данных примера 1



```
stack_class.py
1      5
2      push 1
3      push 2
4      max
5      pop
6      max
```

Рисунок 33 – Файл `input.txt` с данными примера 2



```
stack_class.py
1      2
2      1
```

Рисунок 34 – Файл `output.txt` с результатом работы на данных примера 2

 `stack_class.py`

|   |        |
|---|--------|
| 1 | 3      |
| 2 | push 1 |
| 3 | push 7 |
| 4 | pop    |

Рисунок 35 – Файл input.txt с данными примера 3

 `stack_class.py`

|   |  |
|---|--|
| 1 |  |
|---|--|

Рисунок 36 – Файл output.txt с результатом работы на данных примера 3

 `stack_class.py`

|    |        |
|----|--------|
| 1  | 10     |
| 2  | push 2 |
| 3  | push 3 |
| 4  | push 9 |
| 5  | push 7 |
| 6  | push 2 |
| 7  | max    |
| 8  | max    |
| 9  | max    |
| 10 | pop    |
| 11 | max    |

Рисунок 37 – Файл input.txt с данными примера 4

 `stack_class.py`

|   |   |
|---|---|
| 1 | 9 |
| 2 | 9 |
| 3 | 9 |
| 4 | 9 |

Рисунок 38 – Файл output.txt с результатом работы на данных примера 4

 `stack_class.py`

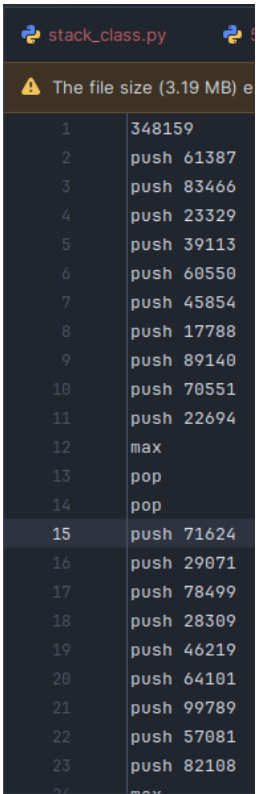
|   |        |
|---|--------|
| 1 | 6      |
| 2 | push 7 |
| 3 | push 1 |
| 4 | push 7 |
| 5 | max    |
| 6 | pop    |
| 7 | max    |

Рисунок 39 – Файл input.txt с данными примера 5

 `stack_class.py`

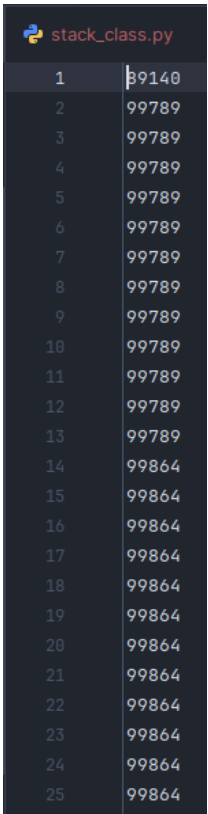
|   |   |
|---|---|
| 1 | 7 |
| 2 | 7 |

Рисунок 40 – Файл output.txt с результатом работы на данных примера 5



|    |            |
|----|------------|
| 1  | 348159     |
| 2  | push 61387 |
| 3  | push 83466 |
| 4  | push 23329 |
| 5  | push 39113 |
| 6  | push 60550 |
| 7  | push 45854 |
| 8  | push 17788 |
| 9  | push 89140 |
| 10 | push 70551 |
| 11 | push 22694 |
| 12 | max        |
| 13 | pop        |
| 14 | pop        |
| 15 | push 71624 |
| 16 | push 29071 |
| 17 | push 78499 |
| 18 | push 28309 |
| 19 | push 46219 |
| 20 | push 64101 |
| 21 | push 99789 |
| 22 | push 57081 |
| 23 | push 82108 |

Рисунок 41 – Файл input.txt с данными сгенерированного примера (348159 команд)



|    |       |
|----|-------|
| 1  | 89140 |
| 2  | 99789 |
| 3  | 99789 |
| 4  | 99789 |
| 5  | 99789 |
| 6  | 99789 |
| 7  | 99789 |
| 8  | 99789 |
| 9  | 99789 |
| 10 | 99789 |
| 11 | 99789 |
| 12 | 99789 |
| 13 | 99789 |
| 14 | 99864 |
| 15 | 99864 |
| 16 | 99864 |
| 17 | 99864 |
| 18 | 99864 |
| 19 | 99864 |
| 20 | 99864 |
| 21 | 99864 |
| 22 | 99864 |
| 23 | 99864 |
| 24 | 99864 |
| 25 | 99864 |

Рисунок 42 – Файл output.txt с результатом работы на данных сгенерированного примера

**Результат работы кода на максимальных и минимальных значениях:**

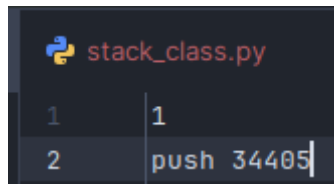


Рисунок 43 – Файл input.txt с вводными данными – минимальными значениями

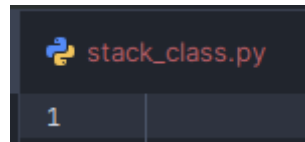


Рисунок 44 – Файл output.txt после выполнения функции с минимальными значениями

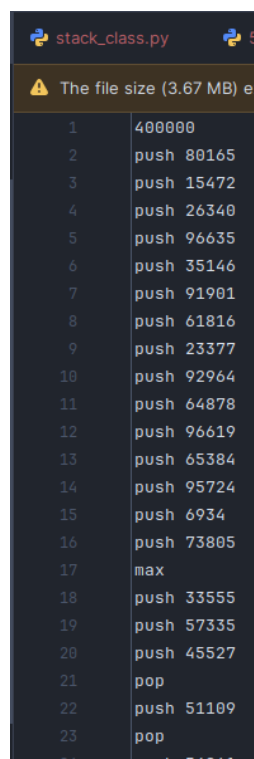


Рисунок 45 – Файл input.txt с вводными данными – максимальными значениями

```

stack_class.py
1 96635
2 96635
3 96635
4 96635
5 96635
6 96635
7 96635
8 96635
9 96635
10 96635
11 96635
12 96635
13 97179
14 97179
15 97179
16 99131
17 99131
18 99131
19 99131
20 99131
21 99131
22 99131
23 99131
24 99131
25 99131

```

Рисунок 46 – Файл output.txt после выполнения функции с максимальными значениями

**Таблица затрат памяти и времени по заданию 5:**

|                                                                   | Время выполнения | Затраты памяти |
|-------------------------------------------------------------------|------------------|----------------|
| Нижняя граница диапазона значений входных данных из текста задачи | 0.0000 сек       | 0.52 KB        |
| Пример 1                                                          | 0.0000 сек       | 0.54 KB        |
| Пример 2                                                          | 0.0000 сек       | 0.54 KB        |
| Пример 3                                                          | 0.0000 сек       | 0.54 KB        |
| Пример 4                                                          | 0.0000 сек       | 0.58 KB        |
| Пример 5                                                          | 0.0000 сек       | 0.76 KB        |
| Собственный пример                                                | 0.2463 сек       | 8.80 MB        |
| Верхняя граница диапазона значений входных данных из              | 0.2823 сек       | 10.14 MB       |

**Вывод по задаче:** Чтобы поддерживать операции push, pop и max за  $O(1)$ , не нужно сортировать стек или копировать его. Достаточно хранить дополнительный стек максимумов. Такой подход даёт амортизированное  $O(1)$  на все три операции и укладывается в ограничения задачи, причём логика полностью опирается на свойства стека: максимум всегда снимается только тогда, когда он оказывается на вершине.

## Задача №7. Максимум в движущейся последовательности

Задан массив из  $n$  целых чисел -  $a_1, \dots, a_n$  и число  $m < n$ , нужно найти максимум среди последовательности ("окна")  $\{a_i, \dots, a_{i+m-1}\}$  для каждого значения  $1 \leq i \leq n - m + 1$ . Простой алгоритм решения этой задачи за  $O(nm)$  сканирует каждое "окно" отдельно.

Ваша цель - алгоритм за  $O(n)$ .

- **Формат входного файла (input.txt).** В первой строке содержится целое число  $n$  ( $1 \leq n \leq 10^5$ ) – количество чисел в исходном массиве, вторая строка содержит  $n$  целых чисел  $a_1, \dots, a_n$  этого массива, разделенных пробелом ( $0 \leq a_i \leq 10^5$ ). В третьей строке - целое число  $m$  - ширина "окна" ( $1 \leq m \leq n$ ).
- **Формат выходного файла (output.txt).** Нужно вывести  $\max a_i, \dots, a_{i+m-1}$  для каждого  $1 \leq i \leq n - m + 1$ .
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.
- Пример:

| input.txt       | output.txt |
|-----------------|------------|
| 8               | 7 7 5 6 6  |
| 2 7 3 1 5 2 6 2 |            |
| 4               |            |

- Есть несколько решений этой задачи. Например:
  - использование очереди на основе двух стеков;
  - использование Dequeue.

### Листинг кода.

```
from collections import deque

from lab3.task1.measurments import measure_performance
from random_array import *

@measure_performance
def dynamic_max(arr, n, k):
    dq = deque()
    res = []

    for i in range(n):
        while dq and arr[dq[-1]] <= arr[i]:
            dq.pop()
        dq.append(i)

        if dq[0] <= i - k:
            dq.popleft()

        if i >= k - 1:
            res.append(arr[dq[0]])
    return res

if __name__ == "__main__":
    generate_array(10**5)
    with open("input.txt") as f:
```

```

size = int(f.readline())
array = list(map(int, f.readline().split()))
window_size = int(f.readline())

maxs = dynamic_max(array, size, window_size)
with open("output.txt", "w") as f:
    f.write(" ".join(map(str, maxs)))

```

### Листинг кода дополнительных функций

```

import random
import numpy as np

def generate_array(n=None, k=None):
    if not n:
        n = random.randint(1, 10 ** 5)
    if not k:
        k = random.randint(1, n)
    fin_arr = np.random.randint(0, 10 ** 5, size=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', f'" ".join(map(str, fin_arr))\n',
f'{str(k)}'])

    return None

```

### Текстовое объяснение решения.

Алгоритм проходит массив слева направо и в deque хранятся индексы тех элементов, которые ещё могут быть максимумами в текущем или будущих окнах. Внутри deque значения по этим индексам всегда убывают. Для этого перед добавлением нового элемента я выкидываю из хвоста все индексы с меньшим или равным значением - они уже никогда не станут максимумом, если справа появился больший элемент. После этого добавляется текущий индекс в конец. С головы deque удаляю индексы, которые «вышли» из окна (их позиция  $\leq i - k$ ). Как только мы дошли до полного окна ( $i \geq k - 1$ ), максимумом в этом окне становится элемент с индексом, лежащим в голове deque, - я просто берусь `arr[dq[0]]` и добавляю в результат.

В итоге каждый индекс попадает в deque не более одного раза и не более одного раза оттуда удаляется, все операции над deque гарантированной сложности  $O(1)$ , поэтому вся функция `dynamic_max` работает за  $O(n)$  по времени и  $O(n)$  по памяти. Это укладывается в ограничения задачи даже при  $n = 10^5$  и даёт асимптотически оптимальное решение по сравнению с наивным перебором внутри каждого окна.

### Результат работы кода на примерах из задания:

```
stack_class.py 7-
1      8
2      2 7 3 1 5 2 6 2
3      4
```

Рисунок 47 – Файл input.txt сданными примера

```
stack_class.py
1      7 7 5 6 6
```

Рисунок 48 – Результат работы кода для примера, записанный в output.txt

```
stack_class.py 7-dynamic_max.py random_array.py measurments.py output.txt input.txt
⚠ This document contains very long lines. Soft wraps were enabled to improve editor performance. Hide notification Don't show aga
1 49330
2 23977 32444 59445 51161 91733 83933 90914 24833 41927 24696 32979 32835 23643 86266 89855 66320 69370 97833 58726 56027
  74323 36026 96004 47192 8335 54466 62088 54089 6531 64743 11937 79766 67890 67837 56269 33747 70938 21449 3727 66960
  46448 70368 90621 14729 90136 83031 86362 24526 51711 91051 23619 82453 53651 85646 67492 25886 85179 68878 5876 57371
  99867 37138 80008 77004 69354 41064 76850 47133 55046 63265 88055 46260 80421 57939 76386 60589 48404 86857 40249
  27277 96659 62028 23541 23346 48478 88183 50448 6786 16051 44381 73823 10885 87148 35233 69102 28657 7215 43776 60416
  69417 50710 41109 35117 60559 10270 68014 59248 97652 56269 19359 15197 96233 20598 88580 97137 63427 25836 44832
  27995 16492 71232 25387 55569 39044 56664 39664 41989 57510 26957 12016 76942 65566 35660 82052 72746 57393 83042
  36440 75906 22168 57969 82866 42134 46486 8629 29335 20619 53054 14874 30684 82353 43113 34169 40340 95911 68292 93358
  49692 48024 8769 86099 32723 42410 95931 10230 54023 88252 62281 59265 50710 12364 39445 94494 63123 65394 64650 8408
  76800 25200 94515 46214 2892 20715 30727 513 44561 12102 26509 55052 92243 42165 37505 47871 35872 29581 13914 35658
  2193 66362 22069 54621 70142 61742 29825 57489 68451 98 16252 37151 68637 99490 98078 13145 60847 33833 57251 39276
  11658 39185 6432 69471 28539 66123 85286 94210 49784 59169 47364 75814 29251 15712 64172 57789 54635 69395 62793 15621
  47788 66493 41956 52828 50078 13320 33803 2721 95189 93521 30238 15885 77389 80220 26109 5405 43139 35470 2760 94399
  87191 58319 70661 16958 82923 2408 79114 36851 1396 79548 60184 32118 7510 57499 13433 74356 59466 46876 13641 18396
  96575 66446 50561 31043 59343 3626 11554 99199 17181 69088 98936 43029 94271 76068 96289 82236 84803 74294 60379 42813
  23810 32909 92658 82359 36803 15652 93849 34242 47131 46611 38664 16654 42103 30595 77105 55301 92487 955 33721 67254
  3159 48125 88650 91065 56502 76277 94284 65757 18379 77426 8132 32629 62824 29533 99786 91611 95271 37279 35145 62443
  64077 22044 90309 18261 69156 48121 58167 24344 13057 967 77576 46998 82244 45573 37737 76981 76547 17003 63584 95438
  13036 85132 66659 39519 18557 22120 95071 1799 14482 96108 3257 19245 90360 47060 43570 58670 81067 72615 88675 62568
  19267 63450 50129 58887 79671 52968 57518 40505 56329 78213 20575 77853 44467 20515 46570 34537 72686 46932 78810
  6230 86800 20687 1652 96902 65842 79138 78115 25028 51720 40030 52485 39837 58658 48416 34945 82398 94346 93688 8632
  66401 19018 81887 21052 86179 41858 20207 15239 45230 31501 23376 17730 81276 14013 70778 13189 72080 64610 8535 87689
  85040 37635 14346 50200 11677 01070 50110 26671 10714 70857 10020 0130 02100 57537 62100 11022 17070 12070 52731
```

Рисунок 49 – Файл input.txt со сгенерированными входными данными (49330 значений, окно 10211)

A screenshot of a code editor interface. At the top, there are tabs for 'stack\_class.py', '7-dynamic\_max.py', 'random\_array.py', 'measurments.py', 'output.txt', and 'input.txt'. The 'output.txt' tab is active. A notification bar at the top states: 'This document contains very long lines. Soft wraps were enabled to improve editor performance.' The main area shows a single line of code consisting of 1000 '9' characters, wrapped across the screen. The line number '1' is visible on the left margin.

Рисунок 50 – Результат работы кода для сгенерированных данных, записанный в output.txt

**Результат работы кода на максимальных и минимальных значениях:**

A screenshot of a code editor showing a file named 'stack\_class.py'. It contains a table with 3 rows and 2 columns. The first column contains the numbers 1, 2, and 3. The second column contains the values 1, 71503, and 1 respectively.

|   |       |
|---|-------|
| 1 | 1     |
| 2 | 71503 |
| 3 | 1     |

Рисунок 51 – Минимальное вводимое значение в input.txt

A screenshot of a code editor showing a file named 'stack\_class.py'. It contains a single row in a table with two columns. The first column contains the number 1, and the second column contains the value 71503.

|   |       |
|---|-------|
| 1 | 71503 |
|---|-------|

Рисунок 52 – Результат работы кода для значения  $n = 1$ , записанный в output.txt

Рисунок 53 – Максимальное вводимое значение в input.txt (окно 29030)

Рисунок 54 – Результат работы кода для значения  $n = 100000$ , записанный в output.txt

Таблица затрат памяти и времени по заданию 7:

|                                                     | Время выполнения | Затраты памяти |
|-----------------------------------------------------|------------------|----------------|
| Нижняя граница диапазона значений входных данных из | 0.0000 сек       | 1.35 KB        |

|                                                                             |            |           |
|-----------------------------------------------------------------------------|------------|-----------|
| текста задачи                                                               |            |           |
| Пример                                                                      | 0.0000 сек | 1.38 KB   |
| Собственный пример                                                          | 0.0382 сек | 344.48 KB |
| Верхняя граница<br>диапазона значений<br>входных данных из<br>текста задачи | 0.0810 сек | 619.63 KB |

**Вывод по задаче:** использование deque для поддержки максимума в скользящем окне позволяет обойтись без пересчёта внутри каждого окна и свести задачу к одному линейному проходу по массиву. Мы храним только нужных «кандидатов в максимум» и за счёт аккуратного удаления устаревших и «слабых» элементов получаем  $O(n)$  по времени при умеренных затратах памяти. Такой подход естественно вписывается в формулировку задачи и является практически оптимальным решением для заданных ограничений.

## Задачи на продвинутый уровень

### Задача №12. Строй новобранцев

В этой задаче  $n$  новобранцев, пронумерованных от 1 до  $n$ , разделены на два множества: *строй* и *толпа*. Вначале *строй* состоит из новобранца номер 1, все остальные составляют *толпу*. В любой момент времени строй стоит в один ряд по прямой. Товарищ сержант может использовать четыре команды. Вот они.

- "I, встать в строй слева от J." Эта команда заставляет новобранца номер I, находящегося в толпе, встать слева от новобранца номер J, находящегося в строю.
- "I, встать в строй справа от J." Эта команда действует аналогично предыдущей, за исключением того, что I встает справа от J.
- "I, выйти из строя." Эта команда заставляет выйти из строя новобранца номер I. После этого он присоединяется к толпе.
- "I, назвать соседей." Эта команда заставляет глубоко задуматься новобранца номер I, стоящего в строю, и назвать номера своих соседей по строю, сначала левого, потом правого. Если кто-то из них отсутствует (новобранец находится на краю ряда), то вместо соответствующего номера он должен назвать 0.

Известно, что ни в каком случае строй не остается пустым. Иногда строй становится слишком большим, и товарищ сержант уже не может проверять сам, правильно ли отвечает новобранец. Поэтому он попросил вас написать программу, которая помогает ему в нелегком деле обучения молодежи и выдает правильные ответы для его команд.

- **Формат входного файла (input.txt).** В первой строке находятся два числа  $N$  ( $1 \leq N \leq 75000$ ) и  $M$  ( $1 \leq M \leq 75000$ ) - количество новобранцев и команд соответственно. Следующие  $M$  строк содержат команды, одна команда на строку. Каждая команда - одна из следующих:

- left I J - соответствует команде "I, встать в строй слева от J."
- right I J - "I, встать в строй справа от J."
- leave I - "I, выйти из строя."
- name I - "I, назвать соседей."

Гарантируется, что все команды корректны, например, leave I не будет заставлять выйти из строя новобранца, стоящего в толпе. Также гарантируется, что строй никогда не будет пустым.

- **Формат выходного файла (output.txt).** Для каждой строки, содержащей name I, выведите в отдельной строке два числа - номера левого и правого соседа новобранца номер I. Если кто-то из соседей отсутствует, выведите ноль вместо его номера.
- Ограничение по времени. Оцените время работы и используемую память при заданных максимальных значениях.
- Примеры:

| input1.txt                             | output1.txt | input2.txt                                        | output2.txt |
|----------------------------------------|-------------|---------------------------------------------------|-------------|
| 3 3<br>left 2 1<br>right 3 1<br>name 1 | 2 3         | 3 4<br>left 2 1<br>right 3 1<br>leave 1<br>name 2 | 0 3         |

### Листинг кода.

```
from nodelist_class import *
from generate_commands import *
from measurments import *

@measure_performance
def build_flow(cmnds, n):
    line = NodeList(n)
    named = []
    for cmnd in cmnds:
        if 'left' in cmnd:
            vali, valj = tuple(map(int, cmnd.strip('left ').split()))
            line.add_node_left(valj, vali)
        elif 'right' in cmnd:
            vali, valj = tuple(map(int, cmnd.strip('right ').split()))
            line.add_node_right(valj, vali)
        elif 'leave' in cmnd:
            val = int(cmnd.strip('leave '))
            line.remove_node(val)
        elif 'name' in cmnd:
            val = int(cmnd.strip('name '))

            named.append(line.name(val))

    return named

if __name__ == "__main__":
    build_commands(n=75000, m=75000)
    with open('input.txt') as f:
        n, cs = tuple(map(int, f.readline().split()))
        commands = []
        for _ in range(cs):
            commands.append(f.readline())

    res = build_flow(commands, n)
    with open('output.txt', 'w') as f:
        for r in res:
            f.write(" ".join(map(str, r)) + "\n")

    print(res)
```

## Листинг кода для дополнительной функции

```
class Node:
    def __init__(self, value):
        self.prev = None
        self.value = value
        self.next = None

    def name_neibs(self):
        left_val = self.prev.value if self.prev else 0
        right_val = self.next.value if self.next else 0
        return left_val, right_val

class NodeList:
    def __init__(self, n):
        self.nodes_list = [None] * (n + 1)
        for i in range(1, n + 1):
            self.nodes_list[i] = Node(i)

        # изначально в строю только 1
        self.head = self.nodes_list[1]
        self.tail = self.head

    def get_node(self, value):
        return self.nodes_list[value]

    def add_node_left(self, val_j, val_i):
        node_j = self.get_node(val_j)
        node_i = self.get_node(val_i)
        prev_left = node_j.prev
        node_j.prev = node_i
        node_i.next = node_j
        node_i.prev = prev_left
        if prev_left:
            prev_left.next = node_i
        else:
            self.head = node_i

    def add_node_right(self, val_j, val_i):
        node_j = self.get_node(val_j)
        node_i = self.get_node(val_i)
        prev_right = node_j.next
        node_j.next = node_i
        node_i.prev = node_j
        node_i.next = prev_right
        if prev_right:
            prev_right.prev = node_i
        else:
            self.tail = node_i

    def remove_node(self, value):
        node = self.get_node(value)
        left = node.prev
        right = node.next

        if left:
```

```

        left.next = right
    else:
        self.head = right

    if right:
        right.prev = left
    else:
        self.tail = left

    node.prev = node.next = None

def name(self, value):
    node = self.get_node(value)
    return node.name_neibs()

```

### Текстовое объяснение решения.

Я реализовала строй как двусвязный список на массивах. Класс Node описывает одного новобранца: он хранит ссылки на предыдущего (prev) и следующего (next) в строю и свой номер (value). Метод name\_neibs по текущему узлу возвращает номера левого и правого соседа, или 0, если сосед отсутствует. Класс NodeList создаёт массив nodes\_list длиной n+1, в котором на позиции i лежит объект Node(i). Это позволяет за O(1) по номеру солдата получить его ноду через get\_node.

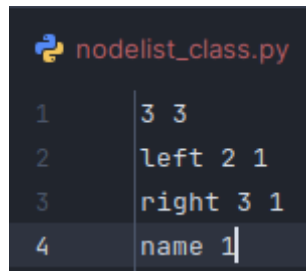
В конструкторе NodeList изначально в строю только новобранец 1: head и tail указывают на ноду с номером 1. Операции add\_node\_left и add\_node\_right реализуют команды «встать слева/справа от J»: по номеру J и I находятся соответствующие ноды, и указатели prev и next перенастраиваются так, чтобы I оказался слева или справа от J. Если вставка происходит на край строя, обновляются head или tail. Метод remove\_node реализует «выйти из строя»: он вырезает ноду из двусвязного списка, соединяя её левую и правую ноды напрямую и обновляя голову или хвост, если удалялся крайний элемент. Метод name просто берёт ноду по номеру и возвращает её соседей через name\_neibs.

Функция build\_flow читает список команд и пошагово применяет их к объекту NodeList. Для left I J и right I J она вызывает соответствующие методы вставки, для leave I - удаление, для name I - запрашивает соседей и добавляет пару чисел в список ответов named. В конце все ответы по командам name записываются в output.txt по одному в строке.

По оценке ресурсов: каждая команда обрабатывается за O(1), так как все обращения к новобранцу идут через прямой индекс в массиве, а связность поддерживается только переназначением нескольких ссылок prev/next. Память - O(N), так как хранится по одному объекту Node на каждого новобранца и несколько ссылок на голову/хвост. Эксперименты на

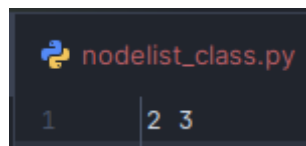
максимальных значениях  $N$  и  $M = 75\,000$  показывают, что программа укладывается и по времени, и по памяти в адекватные рамки.

**Результат работы кода на примерах из задания:**



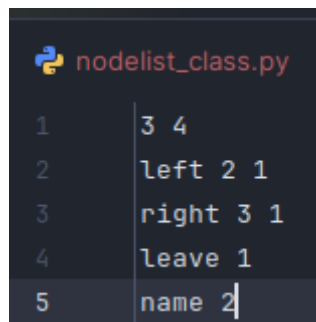
```
nodelist_class.py
1      3 3
2      left 2 1
3      right 3 1
4      name 1
```

Рисунок 55 – Файл input.txt с входными данными из примера 1



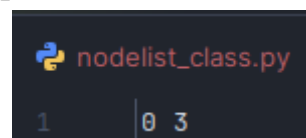
```
nodelist_class.py
1      2 3
```

Рисунок 56 – Результат работы кода для данных из примера 1, записанный в output.txt



```
nodelist_class.py
1      3 4
2      left 2 1
3      right 3 1
4      leave 1
5      name 2
```

Рисунок 57 – Файл input.txt с входными данными из примера 2



```
nodelist_class.py
1      0 3
```

Рисунок 58 – Результат работы кода для данных из примера 2, записанный в output.txt

```

nodelist_class.py 12-buil
1 57914 52967
2 name 1
3 name 1
4 left 36432 1
5 leave 36432
6 name 1
7 left 2555 1
8 name 1
9 left 39920 1
10 name 1
11 name 39920
12 leave 1
13 right 25921 39920
14 leave 2555
15 name 25921
16 name 39920
17 right 35188 39920
18 right 51388 35188
19 right 23462 35188
20 leave 39920
21 leave 25921
22 name 23462
23 leave 35188
24 right 54520 23462
25 right 8492 23462

```

Рисунок 59 – Файл input.txt со сгенерированными входными данными (n=57914, m=52967)

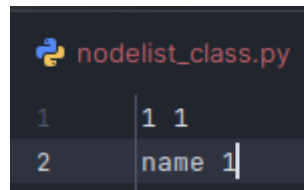
```

nodelist_class.py
1 0 0
2 0 0
3 0 0
4 2555 0
5 39920 0
6 2555 1
7 39920 0
8 0 25921
9 35188 51388
10 54520 51388
11 54520 51388
12 54520 53512
13 30996 0
14 54520 24563
15 9167 7477
16 44616 30996
17 45606 44616
18 23462 8492
19 8492 9167
20 37333 45606
21 14844 8810
22 33765 37333
23 11225 9132
24 6494 37333
25 38711 0

```

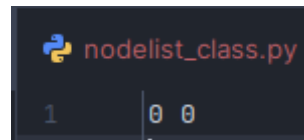
Рисунок 60 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



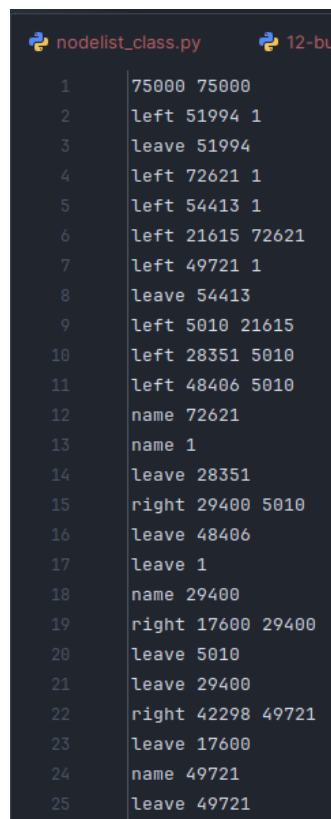
```
nodelist_class.py
1      1 1
2      name 1
```

Рисунок 61 – Минимальное вводимое значение в input.txt



```
nodelist_class.py
1      0 0
```

Рисунок 62 – Результат работы кода для значения  $n, k = 1$ , записанный в output.txt



```
nodelist_class.py 12-bu
1      75000 75000
2      left 51994 1
3      leave 51994
4      left 72621 1
5      left 54413 1
6      left 21615 72621
7      left 49721 1
8      leave 54413
9      left 5010 21615
10     left 28351 5010
11     left 48406 5010
12     name 72621
13     name 1
14     leave 28351
15     right 29400 5010
16     leave 48406
17     leave 1
18     name 29400
19     right 17600 29400
20     leave 5010
21     leave 29400
22     right 42298 49721
23     leave 17600
24     name 49721
25     leave 49721
```

Рисунок 63 – Максимальное вводимое значение в input.txt

```

nodelist_class.py
1      21615 49721
2      49721 0
3      5010 21615
4      72621 42298
5      34011 4425
6      0 43296
7      0 43296
8      43296 0
9      43296 32079
10     0 43296
11     43296 49956
12     70451 0
13     68090 36336
14     0 19659
15     70451 0
16     22283 0
17     0 32310
18     32310 0
19     70451 32310
20     32310 0
21     64481 32310
22     0 7763
23     70451 59160
24     0 42866
25     70451 13311

```

Рисунок 64 – Результат работы кода для значения  $n, k = 75000$ , записанный в output.txt

**Таблица затрат памяти и времени по заданию 12:**

|                                                                    | Время выполнения | Затраты памяти |
|--------------------------------------------------------------------|------------------|----------------|
| Нижняя граница диапазона значений входных данных из текста задачи  | 0.0000 сек       | 0.77 KB        |
| Пример 1 из задачи                                                 | 0.0000 сек       | 1.65 KB        |
| Пример 2 из задачи                                                 | 0.0000 сек       | 1.65 KB        |
| Сгенерированный пример                                             | 0.1915 сек       | 8.14 MB        |
| Верхняя граница диапазона значений входных данных из текста задачи | 0.2439 сек       | 10.65 MB       |

**Вывод по задаче:** наиболее естественной и эффективной моделью строя является двусвязный список, где каждый новобранец знает своих непосредственных соседей. Хранение всех нод в массиве по номеру

позволяет быстро находить нужного солдата по  $I$  или  $J$ , а операции вставки/удаления в таком списке выполняются за константное время. Это полностью удовлетворяет требованиям задачи и масштабируется на максимальные входные данные.

## Задача №13. Реализация стека, очереди и связанных списков

1. Реализуйте стек на основе связного списка с функциями isEmpty, push, pop и вывода данных.
2. Реализуйте очередь на основе связного списка функциями Enqueue, Dequeue с проверкой на переполнение и опустошения очереди.
3. Реализуйте односвязный список с функциями вывода содержимого списка, добавления элемента в начало списка, удаления элемента с начала списка, добавления и удаления элемента *после* заданного элемента (key); поиска элемента в списке.
4. Реализуйте двусвязный список с функциями вывода содержимого списка, добавления и удаления элемента с начала списка, добавления и удаления элемента с конца списка, добавления и удаления элемента *до* заданного элемента (key); поиска элемента в списке.

### Пункт 3. Односвязный список

#### Листинг кода

```
class Node:
    def __init__(self, value):
        self.key = value
        self.next = None

    def name_neibs(self):
        right_val = self.next.key if self.next else 0
        return right_val

class SingleLinkedList:
    def __init__(self, n):
        self.nodes_list = [None] * (n + 1)
        for i in range(1, n + 1):
            self.nodes_list[i] = Node(i)

        self.head = None
        self.tail = None

    def get_node(self, value):
        return self.nodes_list[value]

    def _in_list(self, node):
        if node is self.head or node is self.tail:
            return True
        cur = self.head
        while cur:
            if cur is node:
                return True
            cur = cur.next
        return False

    def _detach(self, node):
```

```

        if not self._in_list(node):
            return

        if node is self.head:
            self.head = node.next
            if self.head is None:
                self.tail = None
            node.next = None
            return

        prev = self.head
        while prev and prev.next is not node:
            prev = prev.next

        if prev is None:
            return

        prev.next = node.next
        if node is self.tail:
            self.tail = prev
        node.next = None

    def show_nodes(self):
        vals = []
        cur = self.head
        while cur:
            vals.append(cur.key)
            cur = cur.next
        return vals

    def find(self, key):
        cur = self.head
        while cur:
            if cur.key == key:
                return cur
            cur = cur.next
        return None

    def add_node_head(self, value):
        node = self.get_node(value)
        if node is self.head:
            return
        if self._in_list(node):
            self._detach(node)

        if self.head is None:
            self.head = self.tail = node
            node.next = None
        else:
            node.next = self.head
            self.head = node

    def remove_node_head(self):
        if not self.head:
            return

```

```

        node = self.head
        self.head = node.next
        if self.head is None:
            self.tail = None
        node.next = None

    def add_node_right(self, key_j, key_i):
        node_j = self.find(key_j)
        if node_j is None:
            return

        node_i = self.get_node(key_i)
        if self._in_list(node_i):
            self._detach(node_i)

        node_i.next = node_j.next
        node_j.next = node_i
        if self.tail is node_j:
            self.tail = node_i

    def remove_node_right(self, key_j):
        node_j = self.find(key_j)
        if node_j is None:
            return

        target = node_j.next
        if target is None:
            return

        node_j.next = target.next
        if target is self.tail:
            self.tail = node_j
        target.next = None

```

## Текстовое объяснение решения

### Структура узла и списка.

Класс Node хранит значение key и ссылку next на следующий элемент; в начале все узлы несвязаны. Класс SingleLinkedList хранит массив узлов для быстрого доступа по ключу и два указателя head и tail на первый и последний элементы списка.

### Проверка принадлежности и вырезание узла.

Метод \_in\_list проверяет, находится ли узел в текущем списке, проходя от head до tail. Метод \_detach аккуратно «вырезает» узел: если он голова, сдвигает head и при необходимости обнуляет tail; если узел в середине или в конце, ищет предыдущий элемент и перенаправляет его next, при необходимости обновляя tail.

### Вывод и поиск.

show\_nodes проходит по цепочке от head по ссылкам next и собирает значения key в список. find линейно ищет узел по значению key, возвращая

ссылку на найденный элемент либо None, если такого элемента в списке нет.

Добавление и удаление в начале.

`add_node_head` берёт узел по ключу, при необходимости вырезает его из прежнего места и вставляет в начало: если список пуст, новый узел становится и головой, и хвостом; иначе его `next` указывает на старую голову, и `head` переносится на этот узел. `remove_node_head` сдвигает `head` на следующий элемент, а если список становится пустым, обнуляет и `tail`.

Добавление и удаление после заданного элемента.

`add_node_right` находит узел с ключом `key_j`, вырезает узел с `key_i`, если он уже в списке, и вставляет его сразу после `node_j`, перенастраивая ссылку `next`; если вставка идёт после хвоста, обновляется `tail`. `remove_node_right` удаляет узел, стоящий сразу после элемента с ключом `key_j`: перенаправляет `next` у `node_j` на следующий за удаляемым, а при необходимости обновляет `tail`.

## Пункт 4. Двусвязный список

### Листинг кода.

```
class Node:
    def __init__(self, value):
        self.prev = None
        self.key = value
        self.next = None

    def name_neibs(self):
        left_val = self.prev.key if self.prev else 0
        right_val = self.next.key if self.next else 0
        return left_val, right_val

class DoubleLinkedList:
    def __init__(self, n):
        self.nodes_list = [None] * (n + 1)
        for i in range(1, n + 1):
            self.nodes_list[i] = Node(i)

        self.head = None
        self.tail = None

    def get_node(self, value):
        return self.nodes_list[value]

    def _in_list(self, node):
        if node is self.head or node is self.tail:
            return True
        if node.prev is not None or node.next is not None:
            return True
        return False
```

```

def add_node_left(self, val_j, val_i):
    node_j = self.get_node(val_j)
    node_i = self.get_node(val_i)

    if not self._in_list(node_j):
        return

    if self._in_list(node_i):
        self._detach(node_i)

    prev_left = node_j.prev
    node_j.prev = node_i
    node_i.next = node_j
    node_i.prev = prev_left
    if prev_left:
        prev_left.next = node_i
    else:
        self.head = node_i

def _detach(self, node):
    if not self._in_list(node):
        return
    left = node.prev
    right = node.next

    if left:
        left.next = right
    else:
        self.head = right

    if right:
        right.prev = left
    else:
        self.tail = left

    node.prev = node.next = None

def remove_node_left(self, value):
    node = self.get_node(value)
    if not self._in_list(node):
        return

    left_node = node.prev
    if not left_node:
        return

    left = left_node.prev
    right = left_node.next

    if left:
        left.next = right
    else:
        self.head = right

    if right:

```

```

        right.prev = left
    else:
        self.tail = left

    left_node.prev = left_node.next = None

def add_node_head(self, value):
    node = self.get_node(value)

    if node is self.head:
        return

    if self._in_list(node):
        self._detach(node)

    if self.head is None:
        self.head = self.tail = node
        node.prev = node.next = None
    else:
        prev_head = self.head
        self.head = node
        node.prev = None
        node.next = prev_head
        prev_head.prev = node

def add_node_tail(self, value):
    node = self.get_node(value)

    if node is self.tail:
        return

    if self._in_list(node):
        self._detach(node)

    if self.tail is None:
        self.head = self.tail = node
        node.prev = node.next = None
    else:
        prev_tail = self.tail
        self.tail = node
        node.next = None
        node.prev = prev_tail
        prev_tail.next = node

def remove_node_head(self):
    if not self.head:
        return
    node = self.head
    self.head = node.next
    if self.head:
        self.head.prev = None
    else:
        self.tail = None
    node.prev = node.next = None

def remove_node_tail(self):

```

```

        if not self.tail:
            return
        node = self.tail
        self.tail = node.prev
        if self.tail:
            self.tail.next = None
        else:
            self.head = None
        node.prev = node.next = None

    def show_nodes(self):
        vals = []
        cur = self.head
        while cur:
            vals.append(cur.key)
            cur = cur.next
        return vals

```

### Текстовое объяснение решения.

Структура узла и списка.

Класс Node для двусвязного списка хранит key, а также две ссылки: prev на предыдущий узел и next на следующий. Класс DoubleLinkedList так же держит nodes\_list и указатели head и tail, задающие границы списка.

Проверка принадлежности и вырезание узла.

Метод \_in\_list определяет, находится ли узел в списке, по указателям head, tail и по наличию связей prev или next. \_detach удаляет узел из существующей цепочки: перекидывает ссылки next у левого соседа и prev у правого, корректируя head и tail, если удаляется крайний элемент; затем обнуляет prev и next у самого узла.

Вывод и поиск (по аналогии).

show\_nodes последовательно проходит от головы по next и собирает значения key. Поиск элемента реализуется аналогично односвязному списку: последовательным обходом от head до tail по next.

Добавление и удаление в начале и в конце.

add\_node\_head и add\_node\_tail перед вставкой при необходимости вырезают узел, чтобы он не остался связан в старом месте. Для добавления в голову новый узел привязывается перед текущей головой (обновляются prev и next), а если список пуст, он становится и головой, и хвостом. Для добавления в хвост узел вставляется после текущего tail и становится новым хвостом. Удаление головы и хвоста (remove\_node\_head, remove\_node\_tail) сдвигает соответствующий указатель на соседний узел, корректирует противоположную ссылку (prev или next) и обнуляет связи у удалённого узла.

Добавление и удаление до заданного элемента.

`add_node_left` вставляет узел `node_i` перед узлом `node_j`: сначала при необходимости вырезает `node_i`, затем связывает его с левым соседом `prev_left` и самим `node_j`, обновляя `head`, если вставка происходит в начало. `remove_node_left` удаляет узел, расположенный слева от заданного: берёт `node.prev`, корректирует связи его соседей и при необходимости переносит `head` или `tail`, очищая `prev` и `nex`t у удалённого элемента.

### **Вывод по задаче:**

В обоих заданиях использован единый подход: список задаётся не массивом, а связями между объектами-узлами; массив `nodes_list` служит только для быстрого получения объекта по ключу. Методы аккуратно поддерживают инварианты структуры (`head`, `tail` и корректные ссылки соседей), обеспечивая добавление, удаление, просмотр и поиск элементов в соответствии с требованиями к односвязному и двусвязному спискам.

## **Вывод**

В лабораторной работе были рассмотрены такие структуры данных, как стек, очередь и связанные списки. Также структуры были применены в прикладных задачах, где показали себя успешно.